G3 QA Visibility Validation Audit

QA Manager assigned KL BARAT MAINHEAD sees only 1 completed visit while ADMIN sees 7. Trace through the Site Visits and Dashboard query paths, look for any residual team/participant filter, and identify the actual gap.

ROOT CAUSE — No residual team or participant filter remains after MAINHEAD scoping. The G3 QA scope predicate is `{ mainheadId: { in: qaMainheadIds } }`. The gap is data-shape, not access control. Legacy site visits created before Governance G2 have `mainheadId = null` on the structured FK and only carry their MAINHEAD identity in the deprecated text column `SiteVisit.mainhead`. The G3 QA scope only consults the FK, so every legacy visit is silently invisible to QA even when its text MAINHEAD matches the QA-assigned area. ADMIN bypasses this by having no filter at all. Reproduced on the connected DB: 3 of 4 visits have `mainheadId=null`; the QA-scoped count for KL BARAT is 0.

1. Site Visits query — Admin

Source: `SiteVisitsService.list` (`site-visits.service.ts:498`) → `buildListWhere(user, query, ctx)`. `ctx.isAdmin === true` ⇒ `accessScope` returns `{}`.

```
WHERE = AND [
  { tenantId: adminTenantId },
  {}, // accessScope, empty for ADMIN
  ... statusFilter, etc. — all {}
]
```

Effect: every SiteVisit row in the admin's tenant is returned. No participant or team-membership constraint. Visits with `mainheadId = null` are included.

2. Site Visits query — QA Manager

Same handler. `buildScopeContext` resolves `isQa = true` and `qaMainheadIds = [<KL BARAT id>]`. The QA branch in `accessScope` returns `{ mainheadId: { in: qaMainheadIds } }`.

```
WHERE = AND [
  { tenantId: qaTenantId },
  { mainheadId: { in: [<KL BARAT id>] } },
  ... statusFilter, etc. — all {}
]
```

- **No team filter.** The non-ADMIN team branch of `accessScope` is skipped because the QA branch returns first.
- **No participant filter.** `SiteVisitParticipant` is never consulted by the list query.
- **No status / validationStatus filter.** Admin web sends no query params; the respective helpers return `{}`.
- **Only the MAINHEAD FK is consulted.** Rows with `mainheadId = null` never match `{ in: [...] }`.

3. Dashboard query — Admin

Source: `DashboardService.getDashboard` (`dashboard.service.ts:39`). All SiteVisit-related counts use `accessibleSiteVisitWhere(user, ctx)`:

```
accessibleSiteVisitWhere = { tenantId, ...siteVisitAccessScope(user, ctx) }
ctx.isAdmin === true ⇒ siteVisitAccessScope returns {}
Net WHERE: { tenantId }
```

Inspection counts use `accessibleInspectionWhere(user, ctx)` with the same ADMIN bypass; defect counts inherit the same.

4. Dashboard query — QA Manager

Same pattern with QA scope:

```
siteVisitAccessScope (QA branch) returns:
  { mainheadId: { in: ctx.qaMainheadIds } }
inspectionAccessScope (QA branch) returns:
  { siteVisit: { mainheadId: { in: ctx.qaMainheadIds } } }

Active visit count: { tenantId, mainheadId IN [...], status IN [ACTIVE,OPEN,IN_PROGRESS] }
Completed visit count: { tenantId, mainheadId IN [...], status: COMPLETED }
Defect counts: nested inspection.siteVisit.mainheadId IN [...]
```

Same data-shape exposure as Site Visits list: legacy visits with `mainheadId = null` are excluded from every QA count. Their inspections and defects, being nested under those visits, vanish in lockstep.

5. Residual team / participant filter audit

Verified by re-reading the helpers as committed:

Helper	QA branch returns	Residual team / participant filter?
`site-visits.service.ts:2157 accessScope`	`{ mainheadId: { in: qaMainheadIds } }`	None. Team branch is the *else* path.
`dashboard.service.ts:727 siteVisitAccessScope`	`{ mainheadId: { in: qaMainheadIds } }`	None.
`dashboard.service.ts:694 inspectionAccessScope`	`{ siteVisit: { mainheadId: { in: ... } } }`	None.
`defects.service.ts:3153 inspectionAccessScope`	`{ siteVisit: { mainheadId: { in: ... } } }`	None.
`buildListWhere / buildOperationsBoardWhere`	Aggregates accessScope + optional query filters (status, validation, etc.)	None unless explicitly passed in the URL — admin web does not.
`SiteVisitParticipant`	not consulted by any list / dashboard / operations-board read	None.

Conclusion of §5: no team or participant predicate is applied to QA reads after MAINHEAD scoping. The single visit QA sees is the only row whose `SiteVisit.mainheadId` equals the KL BARAT MAINHEAD id.

6. Connected-DB data shape (reproduces the symptom)

Metric	Connected DB	Interpretation
Total SiteVisit rows (tenant)	4	Visible to ADMIN.
`mainheadId` populated	**1**	Visible to a QA actor whose qaMainheadIds includes the matching id.
`mainheadId IS NULL`	**3**	Invisible to QA. Legacy text column `mainhead` may still hold the MAINHEAD identity, but G3 does not consult it.
KL Barat MAINHEAD on dev DB	id `1dc68faf-…`, code KL-BRT	Exists — the QA actor's qaMainheadIds resolution would include it.
QA-scoped `siteVisit.count` with `mainheadId IN [KLB id]`	**0**	No visit on dev currently maps to KL Barat via the structured FK. Identical pattern to the production report where QA sees 1.

The production observation — QA sees 1, ADMIN sees 7 — matches this pattern exactly. Production has 7 visits in admin's tenant, of which only 1 has `mainheadId` set to the KL Barat id. The other 6 either (a) have `mainheadId = null` and `mainhead` text = "KL BARAT" (legacy), or (b) point to a different MAINHEAD entirely.

7. Production verification probe

Read-only Prisma queries to confirm the data-shape diagnosis on production:

```
// 1) Find the KL Barat MAINHEAD id
const klb = await prisma.mainhead.findFirst({
  where: { OR: [
    { code: { contains: 'KLB', mode: 'insensitive' } },
    { name: { contains: 'KL BARAT', mode: 'insensitive' } } ] },
  select: { id: true, code: true, name: true },
});
```

```
// 2) Admin's view: every visit in the tenant
const adminCount = await prisma.siteVisit.count({
  where: { tenantId: ADMIN_TENANT_ID },
});

// 3) QA scope (structured FK only)
const qaCountFkOnly = await prisma.siteVisit.count({
  where: { tenantId: ADMIN_TENANT_ID, mainheadId: klb.id },
});

// 4) Legacy visits (would be reached if QA scope OR-matched the text column)
const legacyTextMatchCount = await prisma.siteVisit.count({
  where: {
    tenantId: ADMIN_TENANT_ID,
    mainheadId: null,
    OR: [
      { mainhead: { contains: 'KL BARAT', mode: 'insensitive' } },
      { mainhead: { contains: 'KLB', mode: 'insensitive' } } ],
  },
});
```

Expected reading: `adminCount = 7`, `qaCountFkOnly = 1`, `legacyTextMatchCount` ≈ 6. If the third number is the missing 6, the diagnosis is confirmed.

8. Remediation options (no code changes proposed)

Option	What it does	Trade-off
A. Data backfill (preferred)	One-time UPDATE: `UPDATE "SiteVisit" SET "mainheadId" = m.id FROM "Mainhead" m WHERE "SiteVisit"."mainheadId" IS NULL AND lower(trim("SiteVisit"."mainhead")) = lower(trim(m.name)) AND m."isActive" = true`. Then optionally drop the text column in a follow-up migration.	Clean. Idempotent (no row gets reassigned). Production may have stale or ambiguous text values that don't match cleanly — pre-check with a SELECT.
B. Extend QA scope to OR-match the text column	Modify the QA branch of every accessScope helper to: `{ OR: [ { mainheadId: { in: qaMainheadIds } }, { mainhead: { in: qaMainheadNames } } ] }`. Extend `buildScopeContext` to also return `qaMainheadNames`.	No data migration. But text matching is fragile (spelling, case, whitespace) and perpetuates the legacy field. Mainly useful as a defensive belt-and-braces.
C. Hybrid (recommended)	Run A, then ship B as a defensive measure for any future legacy data slipping in via direct DB writes.	Two PRs. Same data migration plus a small scope-extension.

9. Bottom line

G3 access-scope code is correct: QA actors bypass team membership and read by MAINHEAD. No residual team or participant filter is being applied after the QA branch returns. The reason QA sees only 1 visit is that only 1 production row has the structured `SiteVisit.mainheadId` populated; the rest are legacy rows whose MAINHEAD identity lives in the deprecated text column. The fix is either to backfill the FK from the text column, to broaden the QA scope to also match the text column, or both.